Hydrofrac

Lineage-driven fault injection for Hydro

Hydrofrac checks whether a Hydro program's eventual-consistency claims survive faults. It runs the program in the Hydro simulator, injects crashes and message omissions, and looks for an execution in which surviving cluster members fail to converge.

Hydro already checks an important part of eventual consistency: whether operators produce the same contents when given the same inputs. But that is not the whole problem. A sender can crash halfway through a broadcast. A lossy channel can drop a message to one recipient. The downstream computation may be perfectly deterministic and still leave different members with different contents, because they received different inputs.

Hydrofrac checks this missing part. It finds every collection the program labels `EventualConsistency`, generates a correctness check for each, and uses the program's existing transport declarations to determine which faults are allowed. The test author supplies the topology, workload, and fault bounds. No collection is named and no correctness property is written.

The search is based on Molly's lineage-driven fault injection, or LDFI (Alvaro, Rosen, Hellerstein, SIGMOD 2015). Rather than enumerate faults blindly, hydrofrac works **backwards from successful outcomes**. It records which deliveries contributed to convergence, asks which faults could interrupt those deliveries, and tries those faults next. If the program succeeds through another path, that run supplies another derivation for the search to defeat.

The intended result is a separation of existing Hydro code into two groups:

- **Certified within the test's bounds:** exhaustive search found no allowed fault and schedule that violates the generated EC check.
- **Incorrect under those bounds:** hydrofrac found a fault set and schedule that leave surviving members with different contents at the end of the run.

Not every search reaches one of these conclusions. Sampled schedules, exhausted search budgets, empty probe results, and collections outside the generated specification's class are reported separately. A certificate is not a proof for all inputs, all cluster sizes, or arbitrarily long executions.

This document describes the current implementation. Development history is in `history-hydrofrac-development.md`.

Why Hydro needs hydrofrac

A cluster location has a consistency parameter:

```
Cluster<'a, Tag, Con>
```

The default, `NoConsistency`, allows members to hold different contents. `EventualConsistency` says that members given the same inputs eventually hold the same contents. A collection can acquire that label from:

- A transport, such as `broadcast_closed` using a transport whose `ConsistencyGuarantee` is `EventualConsistency`.
- A manual proof, through `assert_has_consistency_of(manual_proof!(...))`.

- An operator that preserves consistency, such as a `map`, a `filter`, or a `fold` with the required commutativity and idempotence properties.

Collections at a single `Process` do not need this label: there is only one holder.

The type system reasons about how a computation transforms its inputs. It does not establish that a distributed protocol actually gets the necessary inputs to every surviving member.

For example, suppose a process broadcasts two items to three members. Each member inserts received items into a set. Set insertion has the right algebraic properties, but the sender can still die after sending the second item to only two members. No downstream algebra repairs that missing delivery. The protocol needs another dissemination path: an echo, gossip, retransmission, or some equivalent recovery mechanism.

In this document, **complete EC** means both parts of the argument:

1. **Consistent computation:** members that receive the same inputs compute the same contents.
2. **Fault-tolerant dissemination:** allowed faults do not leave surviving members with permanently different inputs to that computation.

Hydro's types address the first part. Hydrofrac tests the second, using the labelled collection's contents as the observable outcome.

“Complete EC” does **not** mean that every item from a failure-free run must survive every fault. If the only sender crashes before sending an item to anyone, EC does not require the system to recover that item. The requirement is convergence among surviving members, not loss-free delivery.

What hydrofrac checks

The generated specification is `Spec::eventual_consistency`. It checks a set-valued collection at a cluster.

First, hydrofrac runs a failure-free probe. It collects every fact observed at any member into a fixed set, `FACTS`. Subsequent runs use that set as the domain of the check.

For each run with faults:

1. Exclude members that the fault hypothesis crashes.
2. For each fact in `FACTS`, determine which surviving members hold it.
3. If any survivor holds the fact, require every survivor to hold it.

For example:

Final contents at three surviving members	Result
`{1, 2}`, `{1, 2}`, `{1, 2}`	Satisfied
`{1}`, `{1}`, `{1}`	Satisfied
`{1, 2}`, `{1, 2}`, `{1}`	Violated
`{}`, `{}`, `{}`	No disagreement; contributes no successful-delivery evidence

The second row is allowed. A fault may have prevented item `2` from entering the system at all. The third row is the bug hydrofrac is looking for: some survivors received `2`, but another did not.

If the probe produces no facts, hydrofrac reports `Vacuous`, not a certificate.

Scope of a certificate

A certificate applies to:

- **One collection.** Every labelled collection gets its own search and its own verdict. Certifying one does not certify another, even one derived from it.
- **One topology and workload.** Cluster sizes, input values, and timer firings come from the test.
- **The declared fault model.** Transport types determine available omissions; the test registers crashable nodes.
- **The declared fault bounds.** These include the crash budget and the end of finite failures, `EFF`.
- **The test's horizon.** The specification is evaluated when the test finishes driving the run.
- **The explored schedules.** A certificate requires `Exploration::Exhaustive`. `Exploration::Fuzz { instances }` produces a sampled result, not a certificate.

The horizon matters. Hydrofrac does not prove unbounded eventual convergence. If a protocol would converge after more retransmission rounds than the test provides, hydrofrac reports the disagreement still present at the test's end. The counterexample establishes failure of the bounded check; deciding whether the protocol is broken or the test is too short requires inspecting that execution.

The fact domain also matters. The generated check ranges over `FACTS`, not arbitrary new facts that appear only in faulty runs. It should not be described as an unrestricted equality check over everything a faulty execution might produce.

Why the specification's form matters

LDFI needs more than an executable predicate. It needs successful derivations whose disruption is necessary for a counterexample. Otherwise the search could overlook a fault that changes the outcome without touching any recorded support.

For a fixed fact and fixed set of correct members, the positive obligation “every correct member holds this fact” is monotone in observed deliveries: adding another observed `(member, fact)` pair cannot undo it. Hydrofrac uses these positive delivery obligations to drive the generated EC search.

This is also why the probe fixes the fact domain. An open-ended requirement such as “every member holds everything that any member holds” can acquire a new obligation when a faulty run produces a new fact. Successful lineage from an earlier run need not explain that fact.

Fixing `FACTS` does not make arbitrary program logic monotone. The certification argument still depends on conservative lineage and sound handling of the generated specification's preconditions and delivery obligations. Hydrofrac does not perform a general syntactic analysis that establishes these properties for arbitrary user-written specifications. Certificates for those specifications are therefore marked **RELATIVE**.

Using hydrofrac

The program does not need fault-injection hooks or observation points. Hydrofrac instruments the network operators already present in its flow and taps the collections it already labels.

The test instantiates the program, asks for a specification over all of its labels, registers the crashable nodes, and drives an execution:

```
let mut flow = FlowBuilder::new();
let source = flow.process::<Source>();
```

```
let members = flow.cluster::<Member>();
let (send, items) = source.sim_input::<u32, TotalOrder, ExactlyOnce>();

// This output is labelled EventualConsistency.
let delivered = reliable_broadcast_closed(items, &members);
delivered.weaken_consistency().sim_cluster_output();

let spec = Spec::all_labels(&mut flow); // Names no collection.

let compiled = flow.sim()
    .skip_consistency_assertions() // The simulator does not yet check them.
    .with_cluster_size(&members, 3)
    .with_provenance()
    .compiled();

let results = Ldfi::new()
    .level(Level::Record)
    .crashable("source", &source)
    .crashable_cluster("member", &members, 0..3)
    .crash_budget(1)
    .spec(&spec)
    .search_all_labels(&compiled, async |_run| {
        send.send(1);
        send.send(2);
        hydro_lang::sim::quiesce().await;
    });
println!("{results}");
```

For this program it prints:

```
EventualConsistency at reliable_broadcast.rs:130 (Cluster(loc2v1), AssertIsConsistent, minted): VIOLATED by {crash
EventualConsistency at reliable_broadcast.rs:156 (Cluster(loc2v1), AssertIsConsistent, minted): VIOLATED by {crash
EventualConsistency at reliable_broadcast.rs:149 (Cluster(loc2v1), Unique, delivered): CERTIFICATE, 10 hypotheses,
```

One line per labelled collection. The first is the source's initial broadcast, violated by the source's crash between two sends. The second is the members' echo broadcast, violated by an echoing member's crash. The third is the delivered set the program exposes, certified: the echo repairs what the sender's crash leaves missing.

`results.verdicts` holds the same information as values, one `LabelVerdict` per collection: `Searched(outcome)` or `NotChecked(reason)`. A test asserts on it per site (`results.at("file.rs:149")`) or overall (`results.all_certified_or_not_checked()`).

To check one collection by hand, build the specification with `Spec::new`, name the collection with `spec.observe_cluster(collection)` and the members with `spec.correct_members(&cluster, 0..n)`, call `spec.eventual_consistency(observed, correct)`, and run `Ldfi::search`. The search is the same.

The body determines which inputs arrive, which timers fire, and how long the protocol has to converge. For protocols that depend on retransmission, gossip, or leader changes, the body must fire the appropriate timers after `run.end_of_finite_failures()`. Ending faults without giving the protocol a recovery window is not a useful convergence test.

Each verdict includes either the faults and failing schedule, or the successful search's scope, grade, hypothesis count, and exploration mode.

Which collections are checked

A **labelled collection** is a node in the program's IR where the `EventualConsistency` label is *minted* or where it *exits* the labelled region.

A label is minted at a node with no labelled input: a `broadcast_closed` over a transport that grants the label, or an `assert_has_consistency_of`. The minting point is where the type system's assumption about faults

lives.

A collection exits the labelled region at a consumer that does not carry the label (`weaken_consistency`, a `batch` into an unlabelled tick) or at a program output. A send to another location is not an exit: the collection continues there under the transport's own rule. The exit is what the program delivers under the label.

Both are reported because they can disagree. Reliable broadcast's initial single-hop broadcast is minted `EventualConsistency` and is violated by the sender's crash. Its delivered set, derived from that label by the type system, certifies anyway, because of the echo the type system cannot see. Only the search distinguishes the two.

The operators between a mint and an exit inherit the label and are not listed. A `Tee` and a label-preserving `Cast` stand for the collection they share. Nodes at the same source site, of the same operator, on the same cluster are one collection; when there is one node per cluster member, the collection is their union.

The generated specification applies to an unbounded stream (or keyed stream, taken as pairs) at a cluster's top level that a network channel feeds. Everything else is reported with a reason and not searched:

- **Local inputs only.** No fault in the model can withhold an input, so there is nothing to search.
- **Not a stream.** A `Singleton`, `Optional`, or `KeyedSingleton` is a lattice value, and the equality variant of the specification is not built.
- **Inside a tick** or an atomic region. The per-tick contents are not the collection's.
- **Excluded by the test** through `Spec::all_labels_except`. Observing a collection shares and serializes its elements, so its element type must be `Clone + Serialize`. Labelled cluster collections nearly always are, being built from network sends; one that is not fails the generated crate's compilation and is excluded by hand.

`Ldfi::search_all_labels` runs one search per checked collection against the shared body. Before each search it tells the observer which collection the search is about and which members of that collection's cluster the hypothesis leaves correct, then clears the fact domain and runs the loop below unchanged. The result is `LabelResults`: one `LabelVerdict` per collection.

How hydrofrac works

The simulator supplies schedules

The Hydro simulator compiles the distributed flow into a single-process executable. Network channels become queues, ticks execute over explicitly scheduled batches, and the test body controls external inputs and timers.

There are two distinct searches:

- Hydrofrac chooses **fault hypotheses**.
- The simulator explores **batchings and orderings** under each hypothesis.

Exhaustive schedule exploration is needed for a certificate. Sampling can find a counterexample, but a successful sample cannot establish that no failing schedule exists.

Transports supply the fault model

Hydrofrac inserts a filter before every network send. The filter can omit messages or simulate a crash.

Omission drops a message identified by its channel, sender, destination, and wire bytes. While finite failures last, every byte-identical copy on that channel is dropped. Retransmitting an identical payload therefore does not create a new omission identity.

Omissions are available on lossy channels such as `lossy_delayed_forever`. A `fail_stop` channel does not permit independent message omissions; delivery failures arise from sender crashes.

Crash is represented as:

```
crash(node after message)
```

The node performs sends through the named message. Later sends are dropped, and its external outputs are suppressed. Naming the crash point by a message rather than a send count makes the hypothesis usable across schedules. It also allows hydrofrac to crash a sender between two sends in a single broadcast fan-out.

Only nodes registered by the test are crashable. Crashes count against the crash budget. Faults take effect before `EFF`; the remaining execution gives the protocol a chance to recover.

The program's transport declarations therefore determine what can go wrong, while the test determines how much can go wrong and when faults must stop.

Lineage explains successful outcomes

Each run records:

- Executions of asynchronous code and ticks.
- Sends and their fates.
- Receives and the deliveries they consumed.
- The executions that produced observed outputs.

Starting from a successful specification output, hydrofrac follows these dependencies backwards to the deliveries that supported it. That backward closure is its derivation.

This is the central LDFI idea. A forward search asks, “What happens if I drop this message?” A lineage-driven search first asks, “Why did this run succeed?” It then chooses faults that could remove the explanation.

Suppose the probe succeeds because a source sends directly to every member. Hydrofrac tries a crash that interrupts that fan-out. If the remaining members repair the missing delivery through an echo, the next successful run records the echo path. A breaking hypothesis must now defeat both known ways of succeeding.

Hydrofrac supports four lineage levels:

Level	Dependencies recorded
`Exec`	Every input to the execution and to earlier executions of the same tick.
`Slice`	The same execution-level information, narrowed by a static slice for each sink. This is the default.
`Record`	Record-level provenance, added by an IR rewrite through `SimFlow::with_provenance`.
`Counterfactual`	Node-local replay with deliveries removed one at a time, to test whether the output depends on them.

More precise lineage can reduce the number of hypotheses, but it is not always available. The record rewrite cannot handle every operator or see through every closure. Counterfactual replay may fail to reproduce a node's original output even when all deliveries are retained; it may also encounter code that panics when a delivery is removed.

When a more precise level cannot produce a usable result, hydrofrac falls back: `Counterfactual` to `Record`, and `Record` to `Slice`.

The direction of this fallback is important. Including an unnecessary dependency gives the search extra faults to try. Omitting a necessary dependency can hide a counterexample. Conservative over-approximation costs search time; under-approximation threatens certification.

The search defeats known derivations

The search follows Molly's loop:

1. Run the failure-free probe, establish `FACTS`, and record the first successful derivation.
2. Convert each derivation into a constraint: a breaking fault set must interrupt at least one delivery supporting it.
3. Find a fault hypothesis that hits every derivation collected so far, subject to the fault bounds.
4. Run the hypothesis under the configured schedule exploration.
5. If a schedule violates the specification, return it as a counterexample.
6. Otherwise, add the successful run's derivation and repeat.

An omission interrupts the matching message. A crash interrupts sends after its crash point. The solver accounts for those effects when constructing a hitting set.

The search stops when it finds a counterexample, when no hitting set remains within the bounds, or when the hypothesis budget runs out. Budget exhaustion is `Outcome::Budget`, not certification.

`Strategy::Blind` searches the same fault universe without lineage, trying untried fault sets in increasing weight order. It is the ablation for measuring what lineage buys. `Ldfi::fuzz_faults_with` instead samples fault sets randomly.

The specification is itself a Hydro flow

`Spec` builds an observer at a dedicated process and compiles it with the program. Before each run, hydrofrac supplies the observer with the probe's `FACTS` and the members not crashed by the hypothesis.

The observer collects distinct `(member, fact)` pairs for correct members, restricted to `FACTS`. Let `PRESENT` be the subset of `FACTS` seen at any correct member. The EC condition is:

```
number of distinct observed (member, fact) pairs
    = number of correct members × number of PRESENT facts
```

This checks that every present fact occurs at every correct member. A fact absent everywhere creates no obligation.

Because the observer is a Hydro flow, its outputs have lineage just like the program's outputs. Hydrofrac uses that lineage to build the search constraints.

`Spec::all_labels` leaves the observer's input as a slot to be filled at compile time. When the flow is compiled for the simulator, a pass over the IR (`sim::observe`) lists the labelled collections and taps each one in the specification's class. The labelled node becomes a `Tee`, as a `.clone()` would make it. The second branch is mapped to `(collection index, bincode of the element)` and sent to the observer over a `fail_stop` channel, exactly the IR a `.send()` to the observer would build. The slot becomes the union of the taps. The pass runs before network compilation and before the provenance rewrite, so the taps are compiled and traced like any other operator and lineage flows through them. A `Tee` branch is invisible to the rest of the program, whose own behaviour is unchanged.

Judgements

The distinction between a certificate and a successful test run should be explicit.

Result	Meaning
Certified	Exhaustive exploration completed without a violation, and no further fault hypothesis remains within the declared bounds.
Violated	An allowed fault set and explored schedule failed the generated check. The result includes the counterexample.
Fuzz result: no violation found	Sampled schedules passed. This is evidence, not a certificate.
Budget exhausted	The search stopped before reaching a conclusion.
Vacuous	The probe supplied no facts to check.
Not checked	The collection is outside the generated specification's class. The reason is reported.

A counterexample does not require exhaustive exploration: one failing execution is enough. A certificate does.

For complete EC, certification combines with the existing type-level reasoning. The type system establishes consistency given the same inputs; hydrofrac supplies bounded evidence that the protocol's fault handling preserves the required dissemination. Neither result substitutes for the other.

Results on existing Hydro code

The existing tests cover every producer of the `EventualConsistency` label in the codebase and the set-valued, cluster-resident labelled outputs of `hydro_std::ec_inference_demos`. Tests are in:

- `hydro_test/src/cluster/ec_ldfi.rs`
- `hydro_test/src/cluster/ec_demos_ldfi.rs`

The searches below allow one crash. Hypothesis counts include the probe. Under `Fuzz`, the count varies by one between runs; where two runs differed, both are given. The multi-Paxos test keeps its proposer correct.

Program	Result
`broadcast_closed`, process → cluster, `fail_stop`	**Violated**, 5 hypotheses. `crash(source after bcast[→1]:Item(2))` interrupts a fan-out.
`broadcast_closed`, process → cluster, `lossy_delayed_forever`	**Violated**, 2 hypotheses. `omit(bcast[→0]:Item(2))` loses an item that is never retransmitted.
`broadcast_closed`, cluster → cluster, `fail_stop`	**Violated**, 5 hypotheses. `crash(source@0 after bcast[0→1]:Item(10))`.
`broadcast_live_from_process`	**Violated**, 5 hypotheses. A source crash interrupts the one-hop broadcast.

Program	Result
`leader_merge_broadcast`	**Violated**, 2 hypotheses. A leader crash interrupts dissemination of `(writer 0, 100)`. The demo documents leader survival as an assumption.
Echo reliable broadcast, `hydro_test` version	**Certified**, 10 hypotheses, exhaustive schedules.
`reliable_broadcast_closed`, `reliable_broadcast_live`	**Certified**, 10 hypotheses each, exhaustive schedules.
`uniform_reliable_broadcast_closed`, thresholds 1 and 2	**No violation found**, 28 hypotheses each, 8 sampled schedules per hypothesis.
`g_set_gossip`, `g_set_gossip_live`	**No violation found**, 13 hypotheses each, 8 sampled schedules per hypothesis.
`multi_paxos`, learners' `learned` output; 1 proposer, 3 acceptors, 3 learners	**No violation found**, 24–25 hypotheses, 8 sampled schedules per hypothesis. Learners rebroadcast learned values; this check exercises that dissemination path.

The distinction is straightforward in this corpus. Single-hop dissemination fails when one crash or omission interrupts delivery. Protocols with echo or gossip mechanisms pass the searches performed so far, with exhaustive certification for the smaller schedule spaces.

This exposes a gap in transport-level justifications for EC. Treating a failed connection as a failed recipient does not account for a sender that dies halfway through a fan-out. Treating a dropped message as indefinitely delayed does not make surviving recipients converge within a recovery window. A transport label alone cannot establish complete EC for the program using it.

Some other labels do not introduce an independent network fault question. Algebraically justified folds preserve consistency given their inputs. `quorum` and `epoch_splice` are EC-preserving combinators whose dissemination assumptions come from their upstream inputs. `synod` has no `EventualConsistency` label. ABD's replica register does have an EC claim, but its lattice `Singleton` output needs a different observer.

Every label of multi-Paxos

`Spec::all_labels` over `multi_paxos` reports ten collections. The run has 1 proposer, 3 acceptors, and 3 learners, all crashable, with one crash allowed and `Fuzz { instances: 8 }`. The body tells proposer 0 to lead round 1, quiesces, sends commands 7 and 8, and quiesces. Line numbers are into `hydro_std/src/ec_inference_demos/multi_paxos.rs` unless named otherwise.

Site	Collection	Result
`:281`, acceptors, minted and delivered	`broadcast_closed` of prepares, proposer → acceptors. Its `.entries()` goes straight into the acceptor's tick batch (`:303`), so the mint is also the exit.	**Violated**, 5 hypotheses. `crash(proposer@0 after :281[0→1]:Ts { round: 1, writer: 0 })`: the proposer dies between the prepares to acceptors 0 and 1.
`:549`, acceptors, minted	`broadcast_closed` of phase-2 accepts, proposer → acceptors.	**Violated**, 8 hypotheses. `crash(proposer@0 after :549[0→1]:(Ts { round: 1, writer: 0 }, 0, 1, Some(8)))`: the accept for slot 1 reaches acceptor 0 only.
`:286`, acceptors, delivered	The `forward_ref` that closes the accept cycle; the same stream as `:549`'s entries, exiting at `use::batch(accepts_fwd, ..)` (`:308`).	**Violated**, 8 hypotheses, by the same crash. Nothing between the broadcast and the batch retransmits.

Site	Collection	Result
`:569`, proposers, minted	`broadcast_closed` of chosen slots, proposer → proposers.	**No violation found**, 17–18 hypotheses, 8 sampled schedules per hypothesis. The proposer cluster has one member, so agreement holds trivially.
`:570`, proposers, delivered	The `.values()` of `:569`, exiting at `weaken_consistency()` (`:571`).	**No violation found**, 19 hypotheses, 8 sampled schedules per hypothesis; trivial for the same reason.
`:582`, learners, minted	`broadcast_closed` of chosen decrees, proposer → learners: the first hop of the learner echo.	**Violated**, 9 hypotheses. `crash(proposer@0 after :582[0→1]:(1, 0, 0, Some(7)))`: slot 0 reaches learner 0 only. This is the premise the echo repairs.
`:591`, learners, minted	`broadcast_closed` of the echo, learners → learners, keyed by echoer.	**Violated**, 10 hypotheses. `crash(learner@0 after :591[0→1]:(1, 0, 0, Some(7)))`: learner 0 dies partway through echoing slot 0, and under its own key that echo is present at some correct learners and absent at others.
`:588`, learners, delivered	`learned = initial.merge_unordered(echo_fwd).unique()`, exiting at the test's `weaken_consistency()`.	**No violation found**, 25–26 hypotheses, 8 sampled schedules per hypothesis. Every learner that receives a decree echoes it to every learner; one crash leaves at least one echoer per decree.
`:608`, learners, delivered	The `SpliceFact` `flat_map_unordered` of `learned`, exiting into the fold at `epoch_splice.rs:190`.	**No violation found**, 25–26 hypotheses, 8 sampled schedules per hypothesis. A function of `learned`, element by element.
`epoch_splice.rs:190`, learners, minted	The `fold` with `commutative = manual_proof!` into `Singleton<SpliceState>`.	**Not checked**: not a stream.

The `.values()` directly after `:281` is a `Cast` and shares `:281`'s collection, so it is not listed. The automatic search of `learned` took 25–26 hypotheses against the manual test's 24–25: under `all_labels` the observer carries every label's facts and selects by index, so the probe's derivation is larger.

The pattern is the one reliable broadcast shows above. A minted single-hop `broadcast_closed` is violated by the sender's crash mid-fan-out (`reliable_broadcast.rs:130`; here `:281`, `:549`, `:582`), because the transport's grant assumes the sender finishes. The delivered set after an echo cycle passes (`reliable_broadcast.rs:149`; here `:588`, `:608`). The receiving side of the echo itself, a cluster → cluster broadcast keyed by echoer (`reliable_broadcast.rs:156`; here `:591`), is violated by an echoer's crash: the echo repairs the merged set, not the keyed record of who echoed. One verdict per label separates the transport's assumption from what the program delivers under it.

Every label of ISR replication, two-phase commit, and ABD

The hand-written harnesses described below also run under `Spec::all_labels` (`isr_replication.rs` and `two_pc_ldfi.rs`, `*_all_labels` tests; `abd.rs` in `hydro_std`). The senders those harnesses hold correct (the client and ZooKeeper; the coordinator) are also run crashable, since a `broadcast_closed` from a process has no premise other than the sender's survival.

Program, fault model	Site	Result
ISR (3 replicas, 4 rounds, `Fuzz { instances: 8 }`), replicas crashable	`:343` write broadcast, client → replicas (minted and delivered); `:360` replication `forward_ref` at the replicas (delivered); `:564` view broadcast, ZK → replicas (minted)	**No violation found**, 4 hypotheses each. No crashable node sends on these channels.
ISR, client and ZK crashable too, `EFF` right after the write	`:343`	**Violated**, 2 hypotheses. `crash(client after write[→1]:Write(1))`.
	`:564`, `:360`	**No violation found**, 5 hypotheses each. Every ZK send lies after `EFF`, so no ZK crash point is admissible within the bounds the body sets.
ISR, client and ZK crashable, `EFF` after the first round	`:343`	**Violated**, 2 hypotheses, by the client's crash as above.
	`:564`, `:360`	**Violated**, 8 hypotheses each. `crash(zk after view[→1]:View { leader: Some(0), isr: [0, 1, 2] })`: ZK dies between two sends of one view, and the replication cycle carries the difference.
2PC with cooperative termination (2 participants, 1 transaction, exhaustive), coordinator and participants crashable	`:86` prepare broadcast (minted and delivered)	**Violated**, 2 hypotheses. `crash(coordinator after prepare[→0]:Txn(1))`.
	`:92` commit broadcast (minted and delivered)	**Violated**, 2 hypotheses. `crash(coordinator after commit[→0]:Txn(1))`.
	`:98` peer `forward_ref`, `:161` peer broadcast	**Vacuous.** The failure-free run carries no peer traffic: the termination timers fire after every decision is known, so no participant queries a peer.
2PC, coordinator held correct	`:86`, `:92`	**Certified**, 3 and 5 hypotheses. `:98` and `:161` vacuous as above.
ABD (3 replicas, 1 client, one write, `Fuzz { instances: 8 }`), replicas and client crashable	`:141` query broadcast, `:150` phase-2 `forward_ref`, `:161` the writes filtered from it, `:314` write broadcast	**Violated**, 2 hypotheses each, by the client's crash between two sends of its query or its write. Every stream the replicas label is a single hop from the client.
	`:165` the register `fold`	**Not checked**: not a stream.

The vacuous rows are statements about the body. A termination protocol whose premise is never exercised in the failure-free run gives the specification nothing to protect, and the search says so rather than certifying. A collection minted inside a tick (`hydro_test`'s `ec_all_labels_classes.rs`: a `batch` of a broadcast into the members' tick, labelled there by `assert_has_consistency_of`) is reported `not checked: inside a tick`, while the broadcast before the tick and the `all_ticks` after it are searched.

Hand-written specifications

Hydrofrac can also check properties that Hydro's types do not express.

A hand-written `Spec` has two Hydro views:

- A **precondition**: if empty, the run is ignored.
- A **postcondition**: if empty while the precondition is nonempty, the run is a counterexample.

`Ldfi` also accepts simpler output goals through `goal`, `goal_all_members`, and `goal_witnessed`.

These interfaces support checks for committed-entry durability in Raft and Paxos, two-phase commit atomicity, ISR replication agreement, quorum counting, and retry-until-ack delivery. Their certificates are marked **RELATIVE**, because hydrofrac does not establish that an arbitrary specification has the form required by the lineage search's completeness argument.

These tests have found substantive defects:

- An injected Raft election-restriction bug, using index-only log comparison, was found in 90 hypotheses with four omissions and no crash. The scenario had not been written down by the author. Correct Raft completed a relative certification search in 235 hypotheses under the same bounds.
- Production Paxos counted P1b replies rather than distinct acceptors. The defect became visible after the test moved `EFF` past the first ballot. Correct Paxos, the Paxos key-value store, and compartmentalized Paxos completed relative certification searches in 372, 301, and 568 hypotheses.
- Simulator exploration also exposed two non-fault defects: `collect_quorum_with_response` was not a function of its input despite its `nondet!` justification, and an idempotence-justified `fold` lacked a simulator implementation of its retry-only cast. Both were fixed.

The larger tests are ignored by default for runtime. The Raft bug and correct-version searches take roughly thirty minutes and an hour; the three corrected Paxos searches take about forty-five minutes together.

Limitations

A certificate covers a test, not all executions. Changing the workload, cluster size, timers, or horizon requires another test. Exhaustive fault search does not generalize those choices.

Sampled schedules do not certify. Uniform broadcast, gossip, and multi-Paxos currently use schedule sampling because their schedule spaces are large.

Convergence is checked at a finite horizon. Hydrofrac cannot distinguish “never converges” from “has not converged yet” using the final contents alone. Test bodies must provide a realistic recovery window.

The generated observer handles set-valued collections. A lattice `Singleton`, such as ABD's max-by-timestamp register, needs an observer that compares final values rather than accumulated facts. That observer is not implemented.

Membership is static in these tests. The simulator starts with all members joined. Tests of `_live` protocols exercise faults, not join timing.

Precise lineage has gaps. The provenance rewrite does not handle `entries_partially_ordered` on keyed streams, so `leader_merge` uses `Level::Slice`. Counterfactual replay failures are treated as inconclusive for the affected node and trigger fallback.

Not every labelled collection is in the class. The automatic specification checks unbounded streams at a cluster's top level that a network channel feeds. Lattice values, collections inside a tick, and collections with local inputs only are reported with the reason and not searched. Observing a collection requires its element type to be `Clone + Serialize`; one that is not must be excluded with `Spec::all_labels_except`. The verdicts do not fail a test by themselves: `LabelResults` is what a test asserts on, and nothing fails for a label the test does not look at.

The fault vocabulary is limited. Byzantine behavior and duplication are not modelled. There is no separate partition primitive, although omission sets on lossy channels can represent partitions. The Raft counterexample

uses four omissions on two links.

Next steps

1. **Add a lattice-aware observer.** Use it to check ABD's `Singleton` register.
2. **Support explicit convergence witnesses.** Protocols that report convergence should be able to use that signal in the specification rather than rely only on a fixed horizon.
3. **Add decision-trace replay.** Record nondeterministic decisions and replay a prefix with one decision changed. This would provide finer counterfactual analysis than node-local replay.
4. **Run Raft's `committed` output under the generated specification.** The harness exists; the search takes about an hour.